



Department of Electronic and Computer Engineering

Electronic Design Project 3A (EDPA301)

Assessment Number 4

Final Report (GROUP)

Internet of Things Wireless Weather Monitoring System

By

Group Number: H		
Student Surname	Initials	Student Number
NZIMANDE	N	22234375
NDABA	S	22262946
PADAYACHEE	D	22201700
SHEZI	SN	21854626

PLAGIARISM DECLARATION

1. We know and understand that plagiarism is using another person's work and pretending it is one's own, which is wrong.
2. This report is my own work.
3. We have appropriately referenced the work of other people we have used.
4. We have not allowed and will not allow anyone to copy our work with the intention of passing it off as their own work.
5. We have not used any form of Artificial Intelligence (AI) to generate this report.
(If it is discovered that AI has been used in more than 20% of the report, we will get a zero mark (0% for the report)).

NZIMANDE

N

Surname and Initials

Student Number



Signature

PADAYACHEE

D

Surname and Initials

Student Number



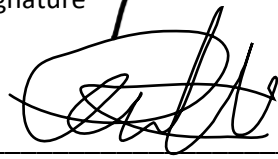
Signature

NDABA

S

Surname and Initials

Student Number



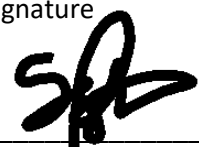
Signature

SHEZI

SN

Surname and Initials

Student Number



Signature

26/05/2025

Date

ABSTRACT

The aim of this project was to design and implement an IoT-based Wireless Weather Monitoring System that tracks temperature, humidity, and light intensity, with automatic and manual control of external devices specifically the fan and lamp, which serve as the controlling elements. The objectives of the system included real-time environmental monitoring, cloud-based storage, intelligent device control using the Blynk IoT platform, and remote data access.

The system was built around an ESP32 microcontroller interfaced with a DHT11 sensor for temperature and humidity, and an LDR module for ambient light detection. Indicator LEDs were used to display temperature conditions locally, while a fan and lamp were controlled via relays based on defined temperature thresholds. An LCD provided on-site readings, while data was sent to Blynk for remote display. The Blynk app was configured to allow switching between automatic and manual modes, with automatic mode adjusting devices based on sensor data and manual mode enabling user control.

Testing confirmed that the system accurately responded to environmental changes, activated control elements appropriately, and updated readings to the Blynk dashboard. The project successfully demonstrated real-time monitoring, basic automation, and wireless communication. Future enhancements will focus on integrating cloud storage for historical data tracking and refining manual control features through the Blynk interface.

Table of Contents

ABSTRACT	iii
List of Figures.....	vi
List of Tables and Equations.....	vii
List of Abbreviations.....	viii
CHAPTER 1: INTRODUCTION	1
1.1 Project objectives	1
1.2 Applications, benefits and limitations of Wireless Weather Monitoring System	2
1.2.1 Applications	2
1.2.2 Benefits.....	2
1.2.3 limitations.....	2
CHAPTER 2: LITERATURE REVIEW.....	3
CHAPTER 3: DESIGN.....	5
3.1 System Operation.....	5
3.2 Circuit Design.....	6
3.3 GUI Design	6
3.4. Cloud Storage	7
3.5. Control strategy.....	8
3.6 Flow chart and code	9
CHAPTER 4: CONSTRUCTION	11
4.1. Prototype.....	11
4.2. Final circuit development.....	12
4.3. Project housing.....	12
5.1. Components Testing.....	13
5.1.1. Blynk Account Setup and Configuration	13
5.1.2. Wi-Fi and Cloud Communication	13
5.1.3. Sensor Integration and Data Reading	13
5.1.4. Gauge Setup and Sensor Display on Blynk	14
5.1.5. Cloud Data Storage (History Graphs)	14

5.1.6. <i>Relay Module and Actuator Testing</i>	14
5.2. Prototype and Final project testing	15
5.2.4. <i>LDR and LED Control for Light Detection</i>	17
CHAPTER 6: RESULTS	18
6.1. Project sensing capabilities	18
6.2. Automatic Control action	18
6.3. Blynk system mode switching.....	20
6.4. Storage data.....	21
6.5. Full System Integration Test	22
CHAPTER 8: TIME MANAGEMENT	24
Chapter 7: CONCLUSION	25
REFERENCES	26
Annexure B. System code.....	28

List of Figures

<u>Figure 3.1. System Block Diagram</u>	5
<u>Figure 3.2 GUI design</u>	7
<u>Figure 3.3. Cloud Storage Graphs</u>	8
<u>Figure 3.4. Feedback control</u>	8
<u>Figure 3.5. Flow Chart</u>	9
<u>Figure 4.1. Prototype</u>	11
<u>Figure 4.2. Project housing</u>	12
<u>Figure 5.1. Blynk setup</u>	13
<u>Figure 5.2. Fan testing</u>	14
<u>Figure 5.3. ESP32 testing</u>	15
<u>Figure 6.1 Sensor display</u>	18
<u>Figure 6.2. Normal Range exceeded</u>	19
<u>Figure 6.3. Temperature below range</u>	19
<u>Figure 6.4. Temperature normal range</u>	20
<u>Figure 6.5. System modes</u>	21
<u>Figure 6.6. Data saved</u>	21
<u>Figure 6.7. Completed system</u>	22

List of Tables and Equations

<u>Table 6.1. system results</u>	23
<u>Table 6.2. results</u>	23
<u>Equation 3.1.</u>	9

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
DHT	Digital Humidity Temperature
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MQTT	Message Queuing Telemetry Transport
Wi-Fi	Wireless Fidelity (Communication Protocol)

CHAPTER 1: INTRODUCTION

Greenhouses are controlled environments used for growing plants, fruits, and vegetables. Maintaining optimal environmental conditions, such as temperature, humidity, and light, is crucial for plant growth and development. Weather monitoring systems play a vital role in ensuring optimal conditions within greenhouses[1].

This project aims to design and develop an automated greenhouse control system that leverages Internet of Things (IoT) technologies, sensor networks, and cloud storage to optimize environmental conditions for plant growth. The system integrates temperature, humidity, and light sensors with an ESP32 microcontroller, relays, and actuators to maintain optimal conditions within the greenhouse. Additionally, the system utilizes BLYNK Cloud storage for data logging and storage, enabling historical data analysis, system performance tracking, and remote monitoring.

The technologies employed in this project have significant relevance to real-world industrial applications. IoT-based automation is increasingly being adopted in precision agriculture, enabling farmers to monitor and control environmental factors, reduce energy consumption, and improve crop yields. Similarly, sensor networks, data analytics, and cloud storage are being used in industrial automation, smart buildings, and environmental monitoring applications. The proposed system demonstrates the potential of IoT technologies in creating smart and sustainable agricultural systems, aligning with the growing demand for efficient and environmentally friendly farming practices[2].

1.1 Project objectives

1. Design and develop an automated greenhouse control system using ESP32 microcontroller that will integrate temperature, humidity, and light sensors with the ESP32 microcontroller for real-time monitoring.
2. Implement IoT-based automation using BLYNK GUI and Wi-Fi protocol for remote monitoring and control.
3. Develop a user-friendly interface using BLYNK GUI and LCD for monitoring and control.
4. Implement automated control strategies for temperature, humidity, and light levels using relays and actuators.

5. Utilize BLYNK Cloud storage for data logging and storage, enabling historical data analysis and system performance tracking.

1.2 Applications, benefits and limitations of Wireless Weather Monitoring System

1.2.1 Applications

- **Greenhouse Automation:** The system can automate temperature, humidity, and light control, optimizing greenhouse conditions for plant growth.
- **Precision Agriculture:** Real-time weather data enables farmers to make data-driven decisions, reducing waste and increasing crop yields.
- **Crop Disease Management:** The system can detect early signs of crop disease, enabling farmers to take proactive measures to prevent disease spread.
- **Smart Farming:** The system can integrate with other smart farming technologies, such as irrigation systems and soil sensor [3].

1.2.2 Benefits

- **Improved crop yield and quality:** Real-time weather monitoring enables farmers to make informed decisions, optimizing crop yield and quality.
- **Increased energy efficiency and cost savings:** Automated weather monitoring systems help reduce energy consumption and costs associated with heating, cooling, and ventilation [4].
- **Enhanced crop disease management:** Real-time weather data enables farmers to take proactive measures to prevent crop diseases, reducing the need for pesticides and other chemicals.
- **Data-driven decision-making:** IoT-based weather monitoring systems provide farmers with real-time data, enabling them to make data-driven decisions and respond quickly to changing weather conditions [5].

1.2.3 limitations

- The project does not detect the rain.
- The project cannot operate without WIFI.
- The project cannot monitor the wind speed and direction.
- The project has no control element for humidity.

CHAPTER 2: LITERATURE REVIEW

Internet of Things Weather monitoring system in a network of sensors and devices such as temperature sensors, humidity, wind, and rain sensors, that collect and transmit real-time weather data. These systems provide numerous benefits to the public such as providing critical weather information for an enhanced emergency preparedness and public awareness [6]. Not only does the systems facilitate the needs of individual users, but it also benefits in working industries, smart homes and in farming applications greenhouse. Greenhouse is an advancement in farming which represents a major step forward in farming technology, allowing for an improvement in growing conditions of vegetables, fruits and crops. Greenhouse also protects plants from adverse atmospheric agents and influences and ultimately modifies the crop microclimate; all this control is reliant on weather manipulation inside the greenhouse [7].

Weather manipulation such as temperature, humidity and wind is critical for influencing plants growth and animal welfare in farming. This is where the need for an IoT weather monitoring system rises, so that it can monitor and collect data from the atmospheric weather and perform a control action to manipulate the weather inside the greenhouse to the desired state that is suitable for plants and animal welfare. Extensive research on weather monitoring system has been carried out in the past and numerous systems has been proposed. Author in [8] proposed a Polyhouse automation system that provides an automatic monitoring and control of the polyhouse environment to farmers in rural locations. The author's system can monitor and control elements of environment such as humidity, temperature and makes use of the sunlight for photosynthesis but the intensity of light is not measured. the system improves the plant's overall development.

Several other systems have been proposed that use IoT on weather monitoring. The weather monitoring systems uses sensors deployed in various locations to capture weather parameters. Most of the systems are integrated with microcontrollers and wireless communication modules for data transmission between the hardware circuit that has sensor and the software applications that are used to read the real time data on the computers or smartphones [9]. The data that is collected by the sensors must be made available through user friendly web applications and mobile interface for an improved accessibility and inclusivity. This is advantageous because it enables everyone to view the weather data from any location without going to the system's location, which can benefit farm workers as well with reading weather data on their mobiles. The technology behind this is Internet of Things (IoT), which is an

advanced and efficient solution for connecting the things to the internet and to connect the entire world of things in a network [10].

Some systems use a serial monitor as a gateway between the sensors and the cloud, where the data is pushed by the sensor on serial monitor which monitors IP address and use HTTP protocol to view data on webserver. This is beneficial for anyone to monitor the weather condition without depending on any applications [11].

The control action for any IoT weather monitoring system depends on the designer's choice and the intended purpose for the designing the system. It depends upon what parameters does the designer wants to control or what solution is the designer trying to make with the designed system.

Some of the problems that were encountered in the previous projects like in author in [12], was the failed communication between the microcontroller and BME280 module, which was resolved by making the two components share the common ground.

The results obtained by author in [13] is that the system sensor capture data and send it to the node MCU controller. The Arduino ide then uploads the sensed data. a serial monitor work as a gateway between the sensor and cloud and the data is pushed by sensor on serial monitor which monitors an IP address. The system displays the data on the webserver and monitor real-time data of weather using environmental parameter and sensor and people can view the data on the webserver using HTTP protocol. The webserver allows everyone to monitor the weather's condition from anywhere. The data is available publicly.

CHAPTER 3: DESIGN

3.1 System Operation

In designing the system plan for the project, **Figure 3.1** represents the operation of the system as a block diagram. The project consists of an ESP32 microcontroller powered by 5V from a laptop USB cable. The USB cable serves as a communication interface between the microcontroller and the laptop with GUI. To measure environmental parameters, a DHT11 sensor, which senses temperature and humidity and LDR, which senses light intensity, were used as inputs to the ESP32 microcontroller. This setup enabled the ESP32 to process and analyse sensor data to the GUI via its Wi-Fi capabilities.

Three LEDs, LED1, LED2 and LED3 are connected to the ESP32 as outputs to turn ON as an indication when the temperature is below the set range, when temperature is within the desired range or when temperature above a set range. LED4 was connected as an output to turn ON in the absence of light and turn OFF when light is detected to save power. 12V fan serves as a cooling element that turns ON if the temperature exceeds the desired range. It is controlled by ESP32 via relay module and powered by an external 12V supply power. The 100W bulb serves as a heating element that turns ON if the temperature drops below the desired range, to heat up the environment. It is powered by an external 230V Eskom supply and controlled by esp32 via relay module.

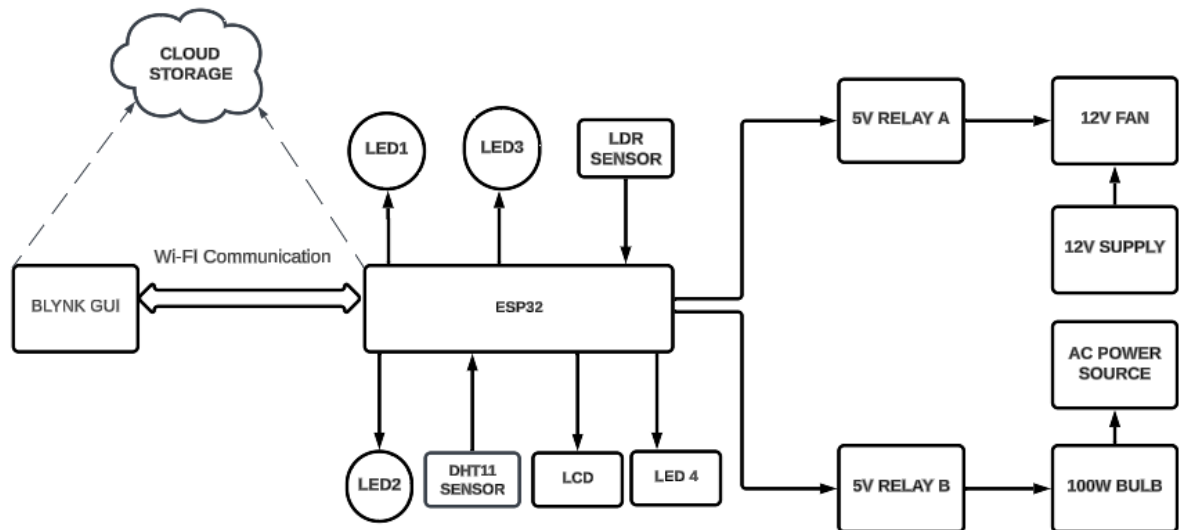


Figure 3.1. System Block Diagram

3.2 Circuit Design

The circuit in **Annexure A**, was designed using Cirkuit Designer. The circuit was designed to sense temperature, humidity and light. A DHT11 sensor was used to sense the temperature and humidity, and LDR was used to sense light. Each sensor was designed to activate a control element based on the sensor readings. The ESP32 microcontroller was utilized to receive and process data from the sensors, generating digital output signals to activate the indicators, and control the corresponding control elements based on the received data. Additionally, the microcontroller was used to transmit data signals to the LCD display for visualization of the DHT11 sensor readings.

The normal operating temperature range for the project was set between (23°C minimum to 28°C maximum). The DHT11 and LDR sensors are connected to the GPIO input pins of ESP32. Indicator LEDs and controlling elements are connected to the GPIO output pins of the ESP32. The fan was set activate automatically if the temperature reading from the sensor exceeds 28°C, for cooling purpose. The red LED2 was set to activate only when the temperature exceeds 28°C. The bulb heater was set to activate automatically if the temperature drops below 23°C, for heating up the environment. The yellow LED3 was set to turn ON only if the temperature is below 23°C. Under normal temperature range, both the fan and heater remain deactivated (OFF), the green LED1 turn ON and remain activated until the temperature drops below or exceeds the normal operating range.

The LDR sensor was set to operate between 0% to 100% of darkness level. The light control, LED4 in Annexure A, was set to turn ON when the darkness reaches 70% and above, to provide light, and turn OFF when the darkness level is below 70% to save power.

100 Ohms resistors were connected in series between the LEDs and ESP32 GPIO pins to limit the current draw and protect the ESP32 from potential damage due to excessive current.

3.3 GUI Design

The implementation of GUI design was done using the Blynk platform, allowing real-time monitoring of environmental data. The gauges for displaying humidity, temperature, and darkness level were included in the interface providing user-friendly and clear visual feedback on each parameter status. This included choosing data streams, setting the range and units for each parameter, and interacting with the system remotely, enhancing usability by providing a centralized view of all monitored data points. The data streams implemented were the temperature display in Celsius degrees, humidity display in percentage unit, darkness level in percentage, system mode as a switch to allow user to switch modes between manual and automatic for the system, ON/OFF control of the fan when system

is in manual mode, ON/OFF control of the heater when system is in manual mode, as displayed in **Figure 3.2**.

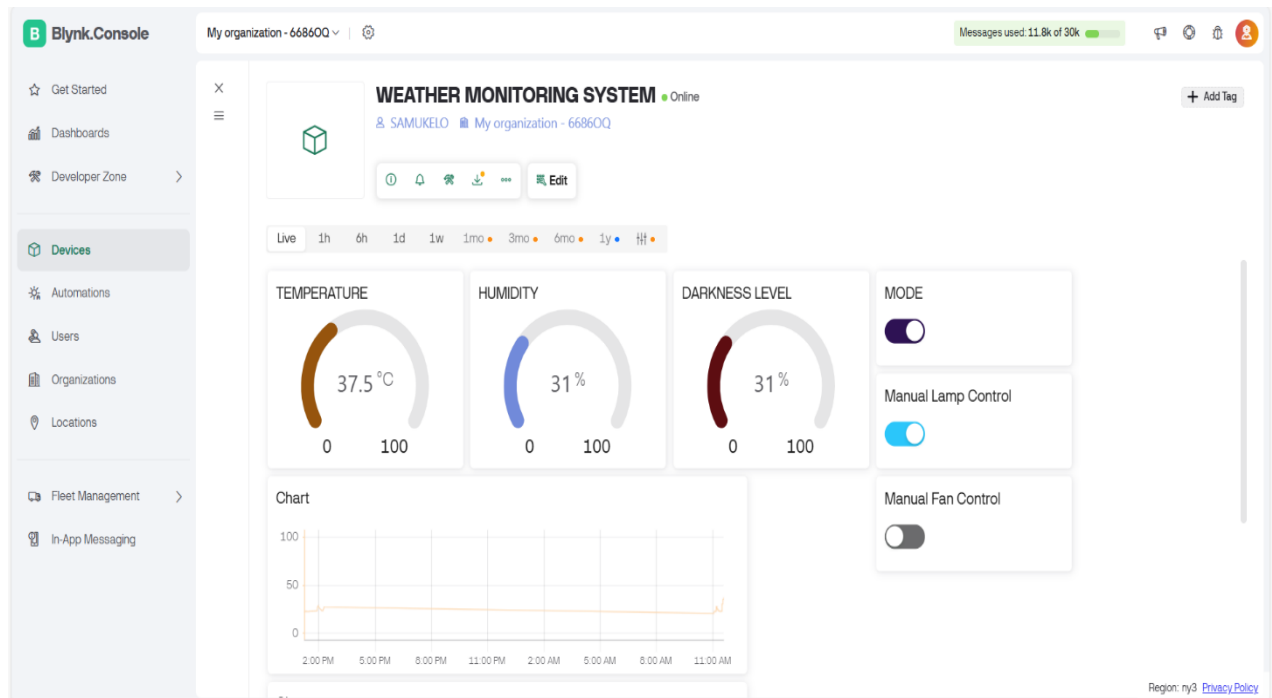


Figure 3.2 GUI design

This design method enables users to monitor the data and choose the mode of operation for the system remotely. Automatic mode operates by the set temperature-based control of the system. Manual mode enables the user to directly activate or deactivate the fan and heater manually, independent of the temperature readings sensed by the system. The system can only work in one mode at a time.

3.4. Cloud Storage

Cloud storage was used in the Blynk application to record and save the historical data, enabling the users to track the data trends for temperature, humidity, and darkness level over time. The cloud storage saves the data recorded only when the system is online. It used charts as graphical representation to shows clearly the data trends across different times of the day as shown in **Figure 3.3**.

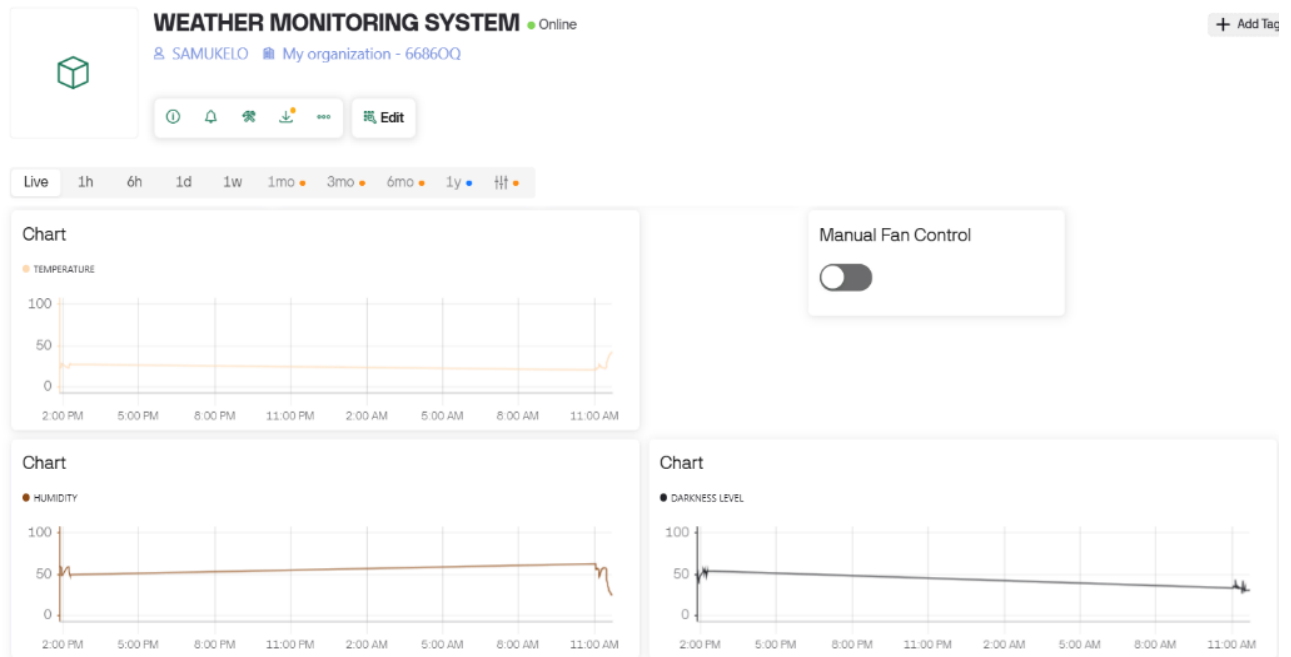


Figure 3.3. Cloud Storage Graphs

3.5. Control strategy

The control system employed a continuous feedback control strategy, calculating the error from the setpoint to adjust and implement corrective actions. Both the temperature and light employed the same feedback control.

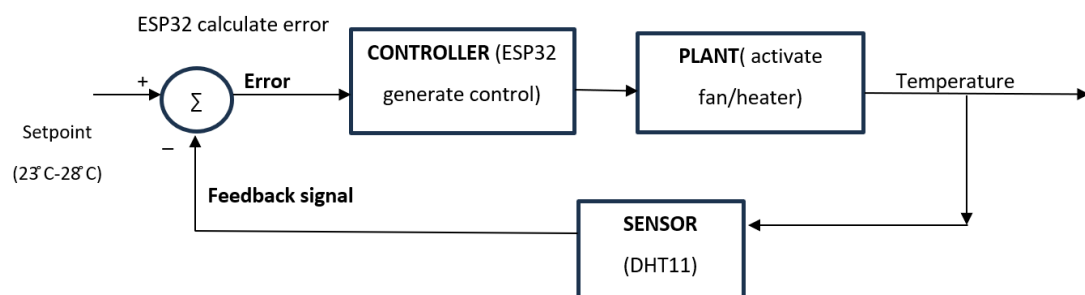


Figure 3.4. Feedback control

Figure 3.4 shows the feedback control that was employed by the system. The DHT11 sensor monitors the temperature and send feedback data to the ESP32. The ESP32 process the data and calculate the deviation of system temperature from the setpoint, an error signal is produced in the ESP32 which

also works as a controller to produce the correction signal that instil direct influence to activate the fan or heater based on the control signal. The controller uses **the formula 3.1**

$$E = SP - Tout \quad (3.1)$$

E = Error

SP = Setpoint, the temperature desired state

Tout = output temperature monitored by DHT11

3.6 Flow chart and code

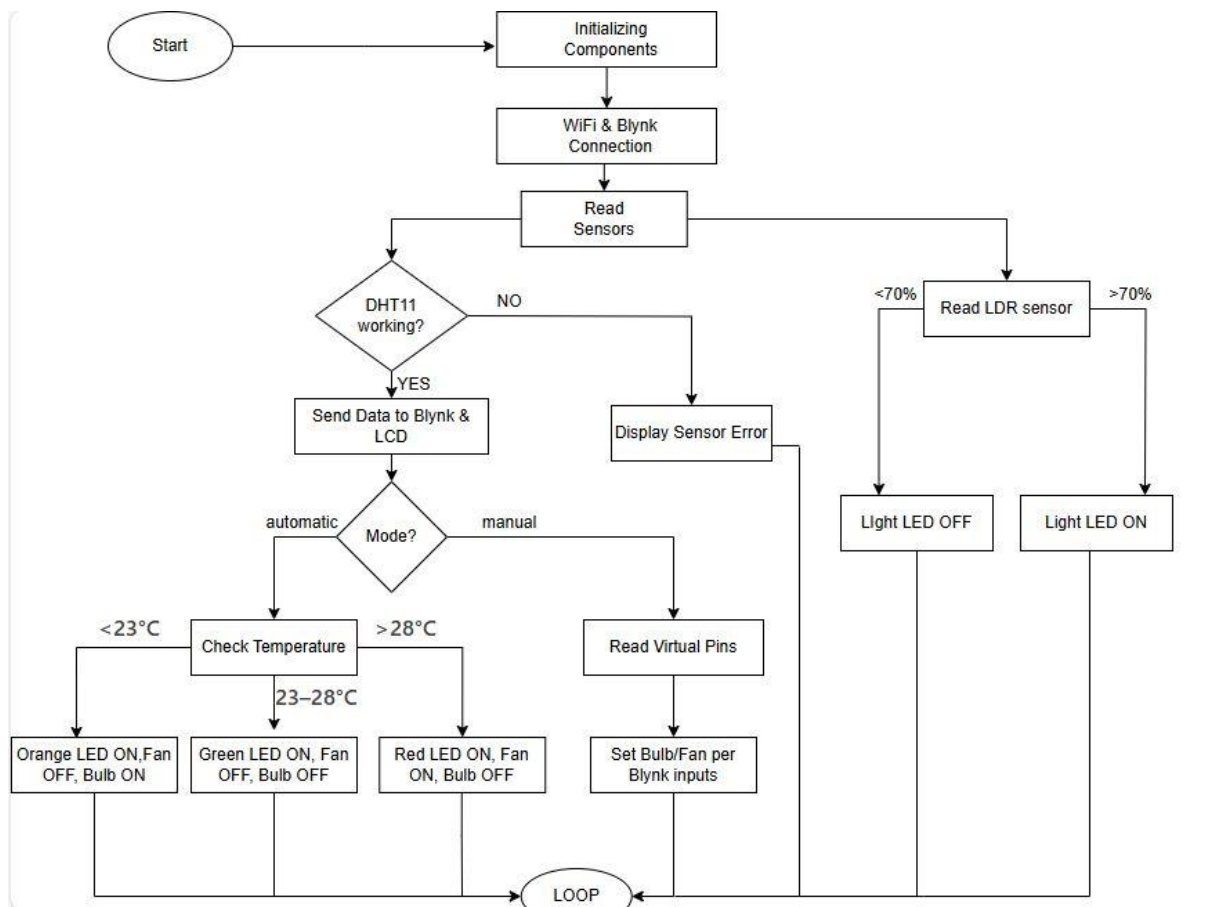


Figure 3.5. Flow Chart

The flowchart, **Figure 3.5** shows the logical organization of an Internet of Things wireless weather monitoring project. Sensors and communication modules were among the first parts to be initialized. a Wi-Fi and Blynk connection were made to allow for remote monitoring and control. After connecting, the system goes into a loop where it gathers information from sensors such the LDR (light intensity) and DHT11 (temperature and humidity). A decision point determines whether the DHT11 sensor is operating properly. an LCD display and the Blynk platform receive the gathered data for real-time visualization. An error message is shown if the sensor malfunctions. The system determines whether to operate in automatic or manual mode. In automated mode, it measures the temperature and, depending on threshold readings, initiates present actions, including turning on or off fans, LEDs, or lightbulbs. In manual mode, users can manually set devices and override automated controls by having the system read virtual pins from the Blynk app. **Annexure B** shows the code of the entire system.

CHAPTER 4: CONSTRUCTION

4.1. Prototype

The circuit prototype was developed and assembled on a breadboard. ESP32 microcontroller, sensors, and other components were connected using breadboard wiring. The ESP32 was powered by a 5V power from the laptop, the 5V pin of ESP32 was connected to another pin of breadboard to make a common point of 5V VCC. The 5V common point was also connecting to the relay modules to allow for relay modules to share common VCC with the ESP32. The 3.3V pin of an ESP32 connects to the VCC pins of the sensors supply input voltage to the sensors. All the components were connected to common ground with esp32. 12V external power supply was connected to the common pin of a relay module to power the fan. The heater was powered by the 230V AC mains supply from the Eskom-connected household outlets through the relay module. 100 Ohms resistors were connected in series between the GPIO output pins of ESP32 and the output LEDs to limit the current draw from ESP32.

The digital data pin of DHT11 was connected to the Esp32 GPIO digital pin 15. The LDR analogue data was connected to the GPIO analogue pin 34 of ESP32. The three LEDs, LED1 (GREEN), LED2 (RED) and LED3 (YELLOW) were connected to the GPIO pin 16, pin 18 and pin 17 respectively. LED4 (WHITE), which was controlled by the LDR, was connected to the GPIO pin 14, in series with a 100Ω resistor from the anode of the LED and the cathode was connected to common ground.

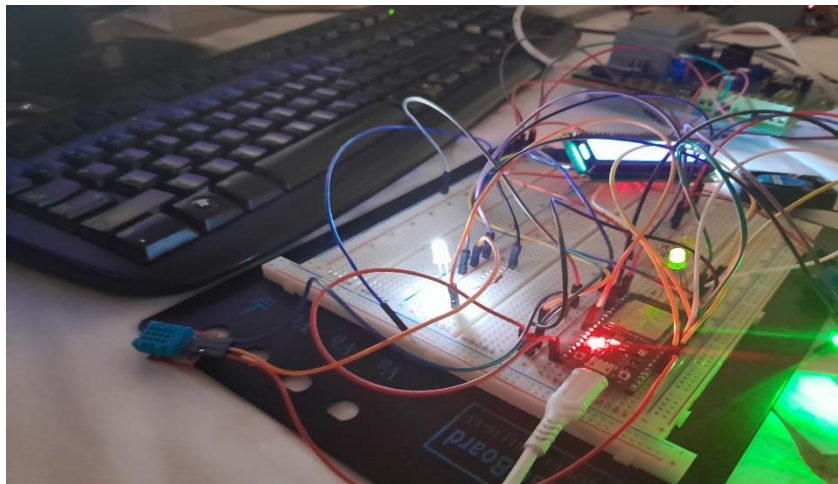


Figure 4.1. Prototype

Breadboard prototype was used for circuit testing, to test the functionality and performance of the project before developing the final project. Various tests were performed under varying weather conditions to test the responsiveness of control elements and performance of the entire project as shown in **Figure 4.1.**

4.2. Final circuit development

In developing of final circuit, the components were soldered onto a Veroboard, following the same routes and connections as was in the prototype. Veroboard was used because of its potential to create strong connections when components are soldered. All the components were soldered into the Veroboard except the sensors, control elements, LCD display, and LEDs, which are connected to the soldered circuit by the joining wires, to enable the sensors and outputs to be mounted in any desired position of the housing. The final project was installed in a project housing. Tests were taken under varying conditions after soldering to check the performance of the project and allow for troubleshooting before the project was installed in a housing.

4.3. Project housing

A housing for the greenhouse project was built. Two enclosures were built, a small housing for the control circuitry and a larger greenhouse structure. The small housing protected the circuit components and displayed key information, including LED indicators, LCD display, and an LDR sensor for ambient light detection. It was mounted on the side of the greenhouse, allowing external visibility of the display, Indicators LEDs, and an LDR sensor. Inside the greenhouse, a DHT11 sensor monitored the temperature and humidity while a heater, cooling fan, and LED lighting were installed for temperature and light control. The greenhouse was made of pine wood and covered with plastic and netting, to enable the farmers to monitor the plant growth from outside. **Figure 4.2** shows the greenhouse in manual mode with an installed display on the wall; the transparent plastic and netting enable observation of plant growth from outside.



Figure 4.2. Project housing.

CHAPTER 5: TESTING

5.1. Components Testing

5.1.1. Blynk Account Setup and Configuration

A project template with virtual data streams was established, to manage sensor readings, control signals, and system modes, along with a new Blynk account. Temperature, humidity, and light level data were assigned to virtual pins V0, V1, and V2, as shown in **Figure 5.1**. Mode switching (Automatic/Manual) was done using virtual pin V3, and manual control of the fan and lightbulb was done with virtual pins V4 and V5. Both the web and mobile dashboards for Blynk had widgets like switches and gauges. To make sure the dashboard was responsive and prepared to receive data from the ESP32, this configuration was tested.

☐	ID	Name	Pin	Color	Data Type	Units	Is Raw	Min
☐	1	TEMPERATURE	V0		Double	°C	false	0
☐	2	HUMIDITY	V1		Double	%	false	0
☐	3	DARKNESS LEVEL	V2		Integer	%	false	0

Figure 5.1. Blynk setup

5.1.2. Wi-Fi and Cloud Communication

Wi-Fi connection between the ESP32 and Blynk Cloud was established. A working password, Blynk authentication token, and SSID were programmed into the ESP32. After uploading the code, the connection status was examined using the serial monitor. The phrase "Blynk Connected" appeared on the monitor, indicating a successful connection. This made sure the ESP32 was prepared to transmit and receive data in real time from the Blynk platform.

5.1.3. Sensor Integration and Data Reading

To enable environmental monitoring, sensors were connected to the ESP32 for performance testing. Temperature and humidity were measured using a DHT11 sensor, while light intensity was detected using an LDR module. The LDR module was tested by exposing it to different light levels and verifying analogue readings via GPIO34. LDR darkness levels were from 0% to 100%, 0% being the brightest and 100% being the darkest, the LED was expected to be off from 0% to 70% and to be on from 71% to 100%, while the DHT11 was verified by tracking temperature and humidity changes on the serial monitor. Accurate and timely data updates were provided by both sensor responses.

5.1.4. Gauge Setup and Sensor Display on Blynk

Sensor readings were sent to Blynk for visualisation after they had been verified. Virtual pins V0 and V1 received the temperature and humidity outputs from the DHT11, and V2 received the LDR readings. The gauge widgets Blynk dashboard were connected to these pins. Changing environmental conditions during testing caused the gauges to update in real time, verifying that the dashboard visualisation and virtual pin communication were working properly. This made it possible for distant users to efficiently monitor environmental conditions.

5.1.5. Cloud Data Storage (History Graphs)

The Blynk history tracking capability was tested to allow for long-term data monitoring. Virtual pins V0, V1, and V2 had the history option activated. While the data changed over time, the system was left to operate for a long time. The constant logging of temperature, humidity, and light levels was then confirmed by accessing historical graphs in Blynk. The precise patterns of the graph demonstrated that Blynk's cloud storage feature was functional.

5.1.6. Relay Module and Actuator Testing

The 2-channel relay module was combined with a 12V fan and a 100W lightbulb to act as heating and cooling components for physical devices. The digital output pins of the ESP32 were linked to the relays. To confirm hardware switching, the relays were first turned on using digital signals that were hardcoded into the ESP32 design. As anticipated, the fan and lightbulb turned on and off, demonstrating that the relay module could manage device management without any problems. The fan is shown in **Figure 5.2** as it is being tested under load.



Figure 5.2. Fan testing

5.1.7. ESP32 Microcontroller

The testing of the ESP32 microcontroller was done with the LED testing. Downloading of drivers was included to ensure the compatibility of the laptop with the board and to read the board by enabling Arduino IDE. The downloading of Arduino WIFI library and integrating it with the code to ensure that the connection protocol (Wi-Fi) can scan and connect with the available hotspot. To complete the testing, switch was created in the blink app, LED was connected to ESP32 microcontroller. Functionality was concluded after the switch and the LED corresponded, LED being off when the switch is in the off state and LED being on when the switch is in the on state. **Figure 5.3** shows complete testing of the ESP32 microcontroller.

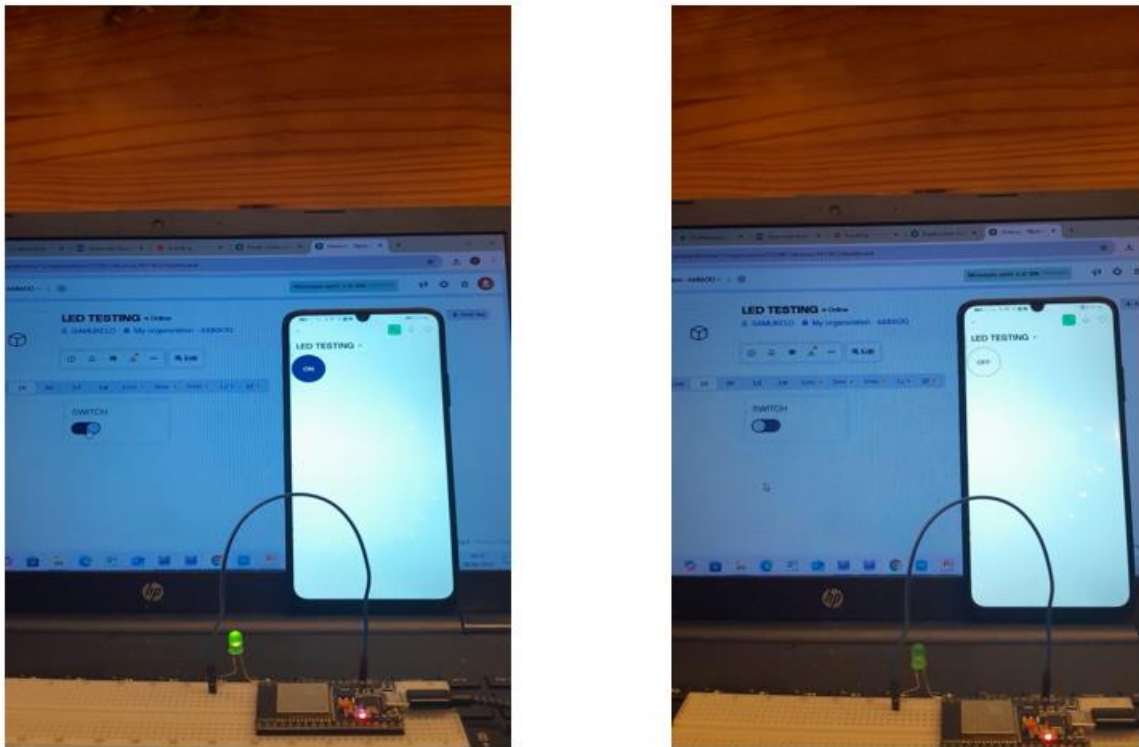


Figure 5.3. ESP32 testing

5.2. Prototype and Final project testing

The project was tested for functionality and performance. Several tests were performed under varying weather conditions, to evaluate the automatic and manual control capabilities of the circuit to the weather conditions. To do prototype testing under different weather conditions, cold air and external heat was used to vary the temperature sensed by the DHT11 sensor, to manipulate temperature to drop below the set range by use of cold air and exceed the set range by application of external heat. For final project testing, the implementation of a manual control option allowed for more accurate

testing circumstances. The greenhouse housing was covered in plastic and sealed, allowing for precise temperature control using manual mode. By activating the fan to cool and the lamp to heat, system could simulate varying weather conditions, enabling for more accurate monitoring and testing in automatic mode.

5.2.1. Temperature testing below optimal range (<23°C)

A test for when the temperature level drops below the desired range, which is any temperature below 23°C. during prototype testing, cold air was used to cool the temperature around the DHT11 sensor until it dropped below 23°C. For final project testing, the system was put in manual mode and the fan was turned on for air cooling, which decreased the temperature rapidly until it dropped below 23°C, Automatic control mode was then enabled, to allow for temperature monitoring and automatic response. At temperature 21.90°C and 30% humidity, the lamp turns on, supplying heat to warm up the environment. The heat produced by the lamp caused an increase to the temperature until the desired range in the greenhouse was maintained. Under this condition, the yellow LED turned on immediately as the lamp turned on. Other LEDs remained off, fan and LED4 remained off.

5.2.2 Temperature testing within optimal range (23°C-28°C)

The system was modified for the normal operating range (23°C – 28°C). When the system was at 24.5°C, the fan and lamp both remained off. Under this condition, the green LED turned on while all the other LEDs remained off. The fan and light bulb remained off. The white LED(LED4) remained off as well because the light was bright. Manual mode was enabled again, and the lamp was manually turned ON for temperature increase. The temperature increased to 27.90°C, and the manual mode was disabled at 27.90°C. At the instance the automatic mode was enabled, the fan and lamp were off and remained off until the temperature reached 28.0°C. The LCD and BLYNK displays continue to display the temperature and humidity outputs.

5.2.3. Temperature testing above optimal range (>28°C)

The testing of the system when the temperature exceeds the normal operating range >28°C. Manual mode was enabled, and the lamp was manually turned ON for temperature increase. The temperature increased to 27.90c, and the manual mode was disabled at 27.90°C. At the instance the automatic mode was enabled, the fan and lamp were off. The temperature increased further up to 80.10°C. When the temperature changed from 28.0°C to 28.10°C , the fan turned on automatically, producing cool air

in the greenhouse. The activation of the fan caused a decrease in temperature, and it decreased until it dropped below 28.0°C. At 28.0°C the fan turned off automatically. Under this condition, the red indicator LED turned on while the other LEDs remained off. The lamp and white LED(LED4) remained off because of high temperature level and bright light intensity in the room.

5.2.4. LDR and LED Control for Light Detection

LDR darkness levels were from 0% to 100%, 0% being the brightest and 100% being the darkest, the LED was set to be off from 0% to 70% and to turn from 71% to 100%. A test under deemed light was carried out, to monitor the responsiveness of circuit to darkness. After the room was darkened, the white LED turned on immediately, providing light. When the lights were turned on again, the white LED(LED4) turned off. Under this condition, the other LEDs can remain on or off, depending on the temperature readings in the display. If it is dark and the temperature is within the desired range, both the white LED(LED4) and green LED turns on. If it is dark and the temperature exceeds 28°C, both the white LED(LED4) and the red LED turn on while the other LEDs remain off. If it is dark and the temperature is below 23°C, both the white LED and yellow LED turn on while the other LEDs remain off.

CHAPTER 6: RESULTS

6.1. Project sensing capabilities

The system features real-time monitoring of temperature and light changes, with indicator LEDs that reflect changes in these parameters. The data is displayed on the LCD, updating dynamically with temperature and light changes. When temperature is below 23°C, the yellow LED turns on, indicating temperature below set range. When temperature is above 28°C, the red LED turns on, indicating temperature above set range. When temperature is within 23°C and 28°C, the green LED turns on, indicating temperature within the set range. The LCD display updates all the changes in real-time. **Figure 6.1** shows the system at temperature 29.10°C, the system displays the sensor data on the LCD display and turns on the red LED as an indication of temperature exceeding normal range.

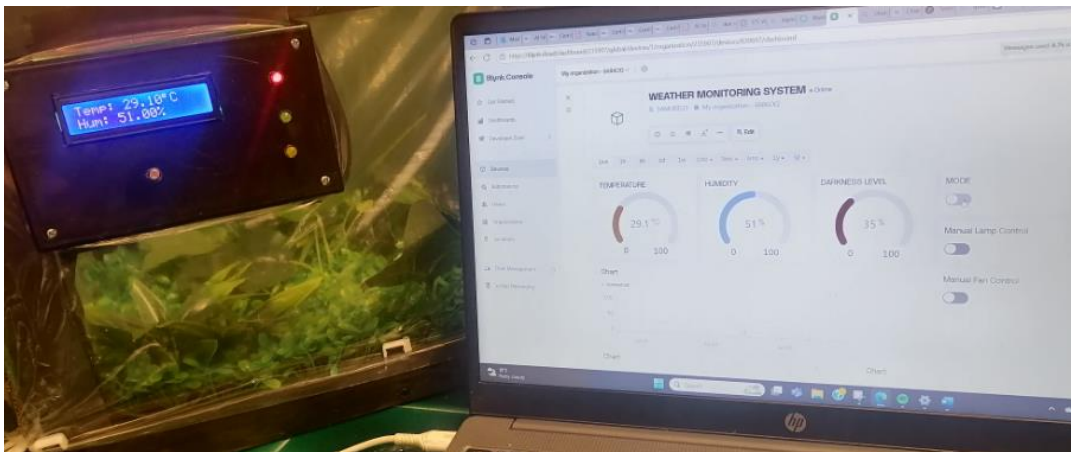


Figure 6.1 Sensor display

6.2. Automatic Control action

The control system of the project responds precisely to sensor data, adjusting to weather changes. Control elements activate and deactivate in response to real-time sensor data. The system accurately adapts to changing weather conditions.



Figure 6.2. Normal Range exceeded

If the temperature sensed by the DHT11 sensor exceeds 28°C , the fan activates automatically producing cooling air to the greenhouse, the cooling air decreases the ambient temperature and maintain it in the desired range. **Figure 6.2.** represents the temperature exceeding the normal range during automatic mode, the fan was turned on. When temperature drops below 23°C , the lamp turns on automatically, producing warmth to the greenhouse. This increases the ambient temperature and maintain the greenhouse temperature within the desired range, **Figure 6.3.** shows the system automatically reacting to temperature drop below normal range at 22.70°C , by turning on the lamp, to increase the temperature.



Figure 6.3. Temperature below range

If the temperature is within 23°C and 28°C, the fan and lamp remain deactivated, this is shown in **Figure 6.4**, where the temperature is at 25.60°C which is within the desired range; under this condition, all the control elements were deactivated. If the light is deemed in the greenhouse, and darkness percentage exceeds 70%, the LED4 turns on automatically to light up the greenhouse, if the darkness percentage drops below 70% or below, the LED4 turns off to save power.

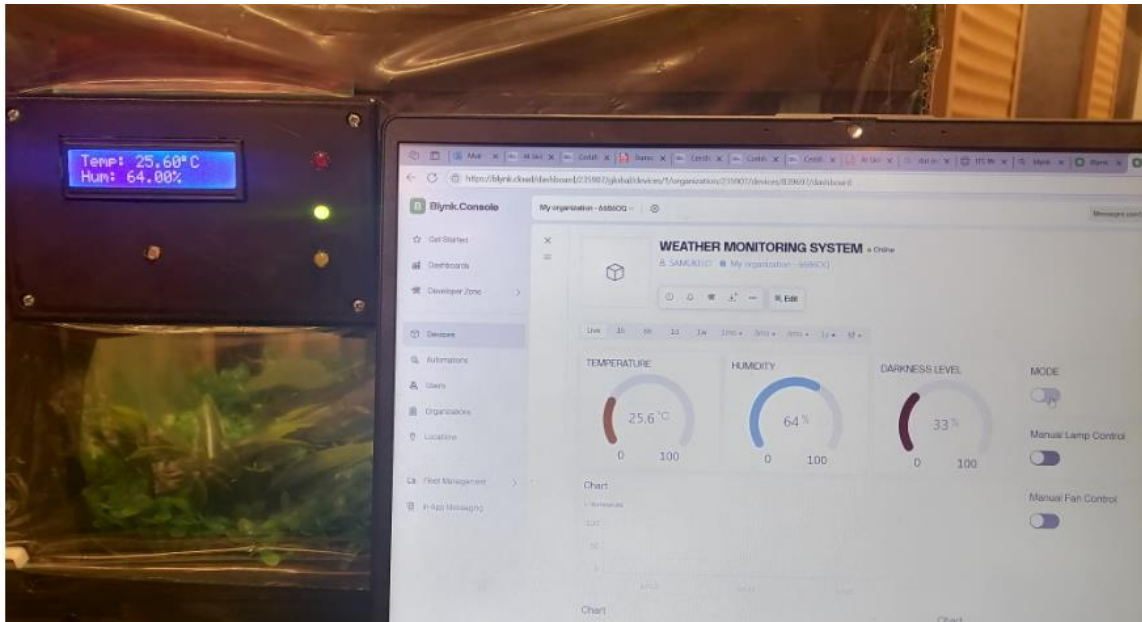


Figure 6.4. Temperature normal range

6.3. Blynk system mode switching

The Blynk application enables remote control of system operation, which enables switching between manual and automatic modes of control. When manual mode is selected, the data-based control automatic mode is disabled. When automatic mode is enabled, the manual mode is disabled. Manual mode allows the user to switch the control elements on and off independent of the sensor data. The fan and lamp can be manually switched on and off at any temperature level, this is shown in **Figure 6.5**, where the manual system mode was enabled on the Blynk application and lamp was manually turned on while the temperature is at 27°C. This proves that the manual control is not data-driven, and it deactivates completely the automatic mode. When the fan is switched on, the temperature in the greenhouse decreases. If the lamp is switched on for a long time, the temperature continuously increases. On manual mode, the fan and lamp can be turned on at the same time.

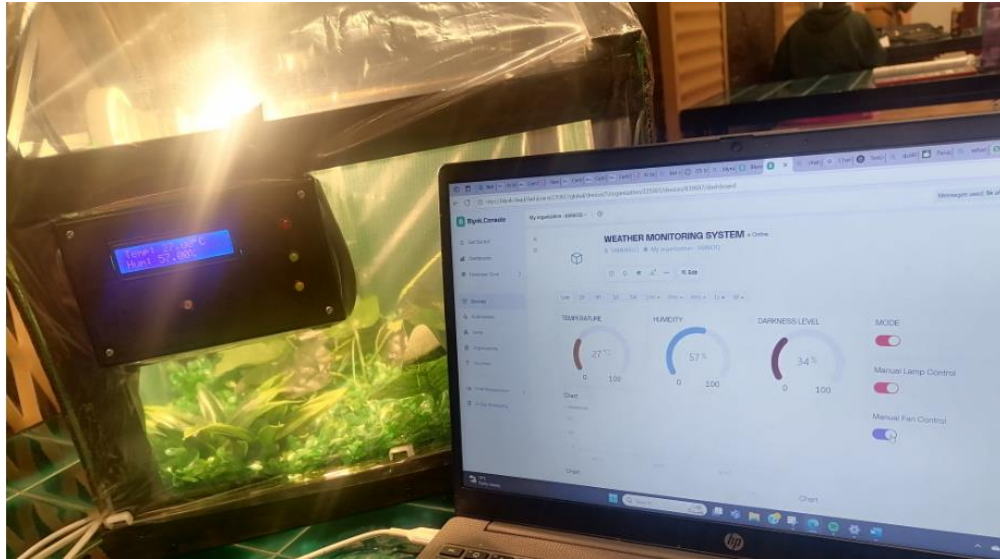


Figure 6.5. System modes

6.4. Storage data

As anticipated, the Blynk History Graph functioned. Test data was produced live after the history for V0 (temperature), V1 (humidity), and V2 (light level) was enabled. These readings were saved by the system, which then appropriately presented them on the graph widgets as shown in **Figure 6.6**. This proved that cloud storage of Blynk feature could be successfully integrated with time-based data tracking.

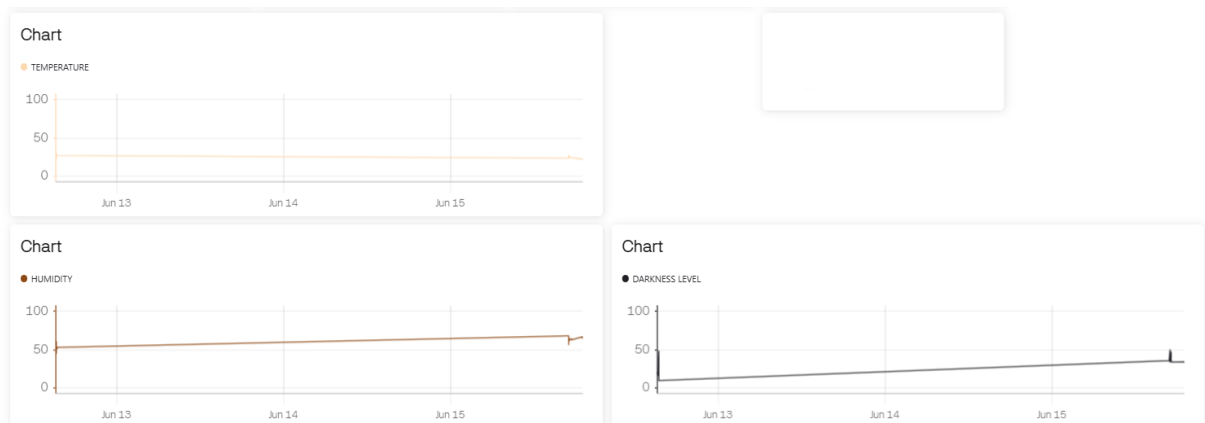


Figure 6.6. Data saved

6.5. Full System Integration Test

The system functioned well after all the parts were connected and powered together. Mode switching functioned, cloud communication was maintained, sensor readings were read and shown, and devices reacted as anticipated as shown in the completed system in **Figure 6.7**. There were no delays, logical mistakes, or hardware problems. The finished system fulfilled all its intended functions and functioned as a coherent whole.



Figure 6.7. Completed system

The operation of the entire system is represented by tables **Table 6.1** and **Table 6.2**

Table 6.1. System results

	LED1(green)	LED2(red)	LED3(yellow)	Fan	Lamp	LED4(white)
Temperature within (22°C-28°C) & Bright light	ON	OFF	OFF	OFF	OFF	OFF
Temperature above 28°C & Bright light	OFF	ON	OFF	ON	OFF	OFF
Temperature below 22°C & Bright light	OFF	OFF	ON	OFF	ON	OFF

Table 6.2. Results

	LED1(green)	LED2(red)	LED3(yellow)	Fan	Lamp	LED4(white)
Temperature within (22°C-28°C) & Deem light	ON	OFF	OFF	OFF	OFF	ON
Temperature above 28°C & Deem light	OFF	ON	OFF	ON	OFF	ON
Temperature below 22°C & Deem light	OFF	OFF	ON	OFF	ON	ON

CHAPTER 8: TIME MANAGEMENT

For the IoT Wireless Weather Monitoring System to be developed successfully, time management was essential. The 84-day project progressed through planning, research, programming, testing, and reporting phases according to a planned timeline. By meeting important deadlines, including proposal submission, firmware completion, successful sensor integration, and final testing, the project was able to remain on schedule.

Following the schedule made sure that every system component was produced and validated completely and on time. Reaching milestones provide precise benchmarks for assessing advancement, seeing problems early, and making the required corrections. In addition to demonstrating effective project management, finishing the project on schedule guaranteed preparedness for final presentations and evaluations. Overall, the project's functionality, success, and general quality were all directly impacted using strict time management.

Annexture represents the group Gantt chart and individual Gantt charts used throughout the making of the project. The Gantt charts were used to manage time for the project.

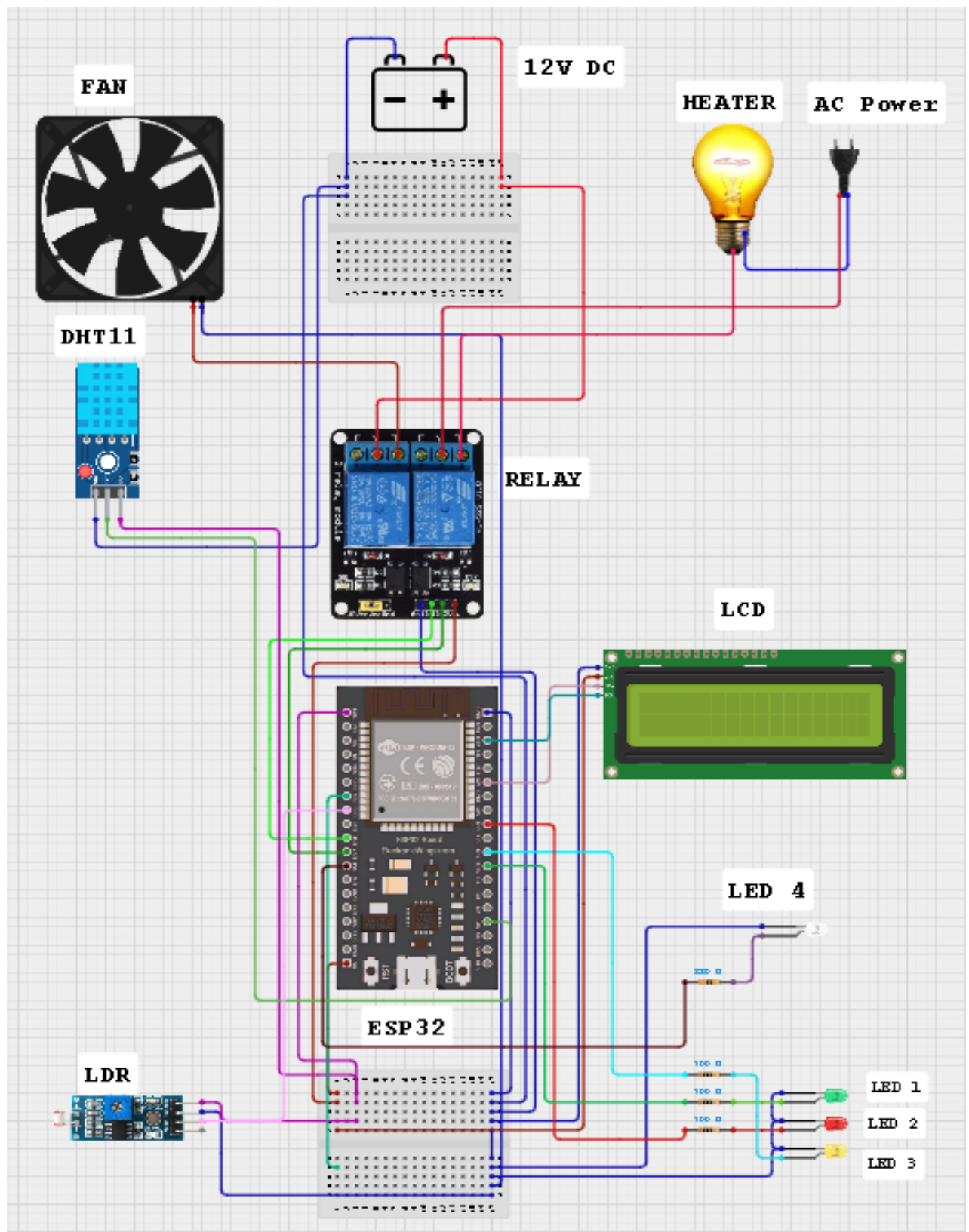
Chapter 7: CONCLUSION

This project successfully developed a system that monitors and controls weather conditions in a greenhouse, with features including real-time data display, automated control, and remote monitoring capabilities via Blynk. The system supports both manual and automatic control modes, allowing for flexible operation and remote access. The system meets all requirements, featuring accurate remote access and control, precise responses to weather changes, and reliable sensor data. It's an ideal solution for automated greenhouses, providing efficient monitoring and control. The system utilizes cloud storage to log and save historical data, enabling trend analysis and informed decision-making. For future recommendations, the systems should be able to operate without WI-FI and the system should be able to monitor and save data on cloud storage even if it is offline.

REFERENCES

- [1] M. MABUNDA, "DRY GAIR," [Online]. Available: <https://drygair.com/blog/what-is-controlled-environmentagriculture/#:~:text=Controlled%20environment%20agriculture%20refers%20to,%2C%20aquaponics%2C%20and%20vertical%20farms..> [Accessed 15 March 2025].
- [2] K. DAVDA. [Online]. Available: <https://oizom.com/what-is-iot-weather-monitoring-systems/>. [Accessed 20 March 2025].
- [3] "WEBBY LAB," [Online]. Available: <https://webbylab.com/blog/weather-reporting-system-using-iot-benefits-use-cases/>. [Accessed 20 March 2025].
- [4] R. SENSOR, "Hunan Rika Electronic," [Online]. Available: <https://www.rikasensor.com/the-benefits-of-wireless-temperature-and-humidity-monitoring-systems.html>. [Accessed 19 March 2025].
- [6] P. Sharma, S. Prakash , "Real Time Weather Monitoring System Using Iot," *ITM Web Conf*, p. 10, 2021.
- [7] G. Vox, M. Teitel, A. Pardossi, A. Minuto, E. Schettini, F. Tinivella, "Sustainable Greenhouse Systems," p. 79, 2010.
- [8] A. Raut, A. Sakhare, S. Dongre, "Weather monitoring and control system for polyhouses using IoT," vol. 2424, p. 3, 2022.
- [9] S. Ogunbunmi, A.A. Taiwo, J.B. Oladosu, H. Sanusi, F.A. Inaolaji, G. Olasunkunmi, "Internet of things weather monitoring system," *World Journal of Advanced Research and Reviews* , vol. 2, p. 12, 2024.
- [10] B.S. Rao, Prof. Dr. K.S. Rao, N. Ome, "Internet of Things (IOT) Based Weather Monitoring System," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, p. 8, 2016.
- [11] P. Horowitz, "PROJECTS FACTORY," [Online]. Available: <https://projectsfactory.in/product/iot-weather-monitoring-system-using-arduino/>. [Accessed 20 March 2025].
- [12] A. Stepanets, "GANTTPRO," [Online]. Available: <https://blog.ganttpro.com/en/how-to-make-a-gantt-chart-in-ms-project/>. [Accessed 19 March 2025].
- [13] R. Sawkare, "VAYUYAAN," [Online]. Available: <https://vayuyaan.com/blog/weather-monitoring-system-using-iot/?srsId=AfmBOornBCU7JTEEnZ6nGf-tE2cLDGQ2-XYMu3fBlg69CW76dixb0Ckl>. [Accessed 19 March 2025].

Annexure A. Circuit diagram



Annexure B. System code

```
#define BLYNK_TEMPLATE_ID "TMPL2gYQd_hHg"
#define BLYNK_TEMPLATE_NAME "WEATHER MONITORING SYSTEM"
#define BLYNK_AUTH_TOKEN "m9IQj94-hwQ0ny_-0Rz1RqYE_MFFIePI"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Wi-Fi and Blynk credentials
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "SAMUKELO";
char pass[] = "SAMUKELO117";

// DHT Settings
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// LCD Settings
LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C address and size

// Virtual Pins
#define VIRTUAL_TEMP V0
#define VIRTUAL_HUM V1
#define VIRTUAL_LDR V2
#define VIRTUAL_MODE V3 // Mode: 0 = Auto, 1 = Manual
#define VIRTUAL_BULB V4 // Manual control for bulb
#define VIRTUAL_FAN V5 // Manual control for fan

// LED Pins (indicator LEDs)
#define ORANGE_LED 25 // Below 23°C
#define GREEN_LED 18 // Between 23-28°C
#define RED_LED 5 // Above 28°C

// Relay control pins
#define LAMP_RELAY 17 // Lamp relay
#define FAN_RELAY 16 // Fan relay

// LDR and light LED
#define LDR_PIN 34 // A0 pin from LDR to GPIO34
#define LIGHT_LED_PIN 27 // LED to light up when dark

int mode = 0; // Default to Auto mode
int manualBulb = 0; // State of bulb in manual mode
int manualFan = 0; // State of fan in manual mode
```

```

void setup() {
  Serial.begin(115200);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Initializing...");

  pinMode(BLUE_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(LAMP_RELAY, OUTPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(LIGHT_LED_PIN, OUTPUT);

  dht.begin();
  Blynk.begin(auth, ssid, pass);

  delay(2000);
  lcd.clear();
}

void loop() {
  Blynk.run();
  sendSensor();
  checkLightLevel();
  delay(2000);
}

void sendSensor() {
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) {
    Serial.println("Failed to read from DHT sensor");
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error    ");
    return;
  }

  // Send to Blynk
  Blynk.virtualWrite(VIRTUAL_TEMP, temp);
  Blynk.virtualWrite(VIRTUAL_HUM, hum);

  // Display on LCD
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temp);
  lcd.print((char)223);
}

```

```

lcd.print("C ");

lcd.setCursor(0, 1);
lcd.print("Hum: ");
lcd.print(hum);
lcd.print("% ");

if (mode == 0) {
  // AUTOMATIC CONTROL
  if (temp < 23) {
    digitalWrite(BLUE_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(LAMP_RELAY, HIGH);
    digitalWrite(FAN_RELAY, LOW);
  } else if (temp >= 23 && temp <= 28) {
    digitalWrite(BLUE_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(LAMP_RELAY, LOW);
    digitalWrite(FAN_RELAY, LOW);
  } else {
    digitalWrite(BLUE_LED, LOW);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(LAMP_RELAY, LOW);
    digitalWrite(FAN_RELAY, HIGH);
  }
} else {
  // MANUAL CONTROL
  digitalWrite(BLUE_LED, LOW);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(LAMP_RELAY, manualBulb);
  digitalWrite(FAN_RELAY, manualFan);
}
}

void checkLightLevel() {
  int ldrValue = analogRead(LDR_PIN);
  int brightnessPercent = map(ldrValue, 0, 4095, 0, 100);

  Blynk.virtualWrite(VIRTUAL_LDR, brightnessPercent);

  if (brightnessPercent < 70) {
    digitalWrite(LIGHT_LED_PIN, LOW); // Bright
  } else {
    digitalWrite(LIGHT_LED_PIN, HIGH); // Dark
  }
}
}

```

```

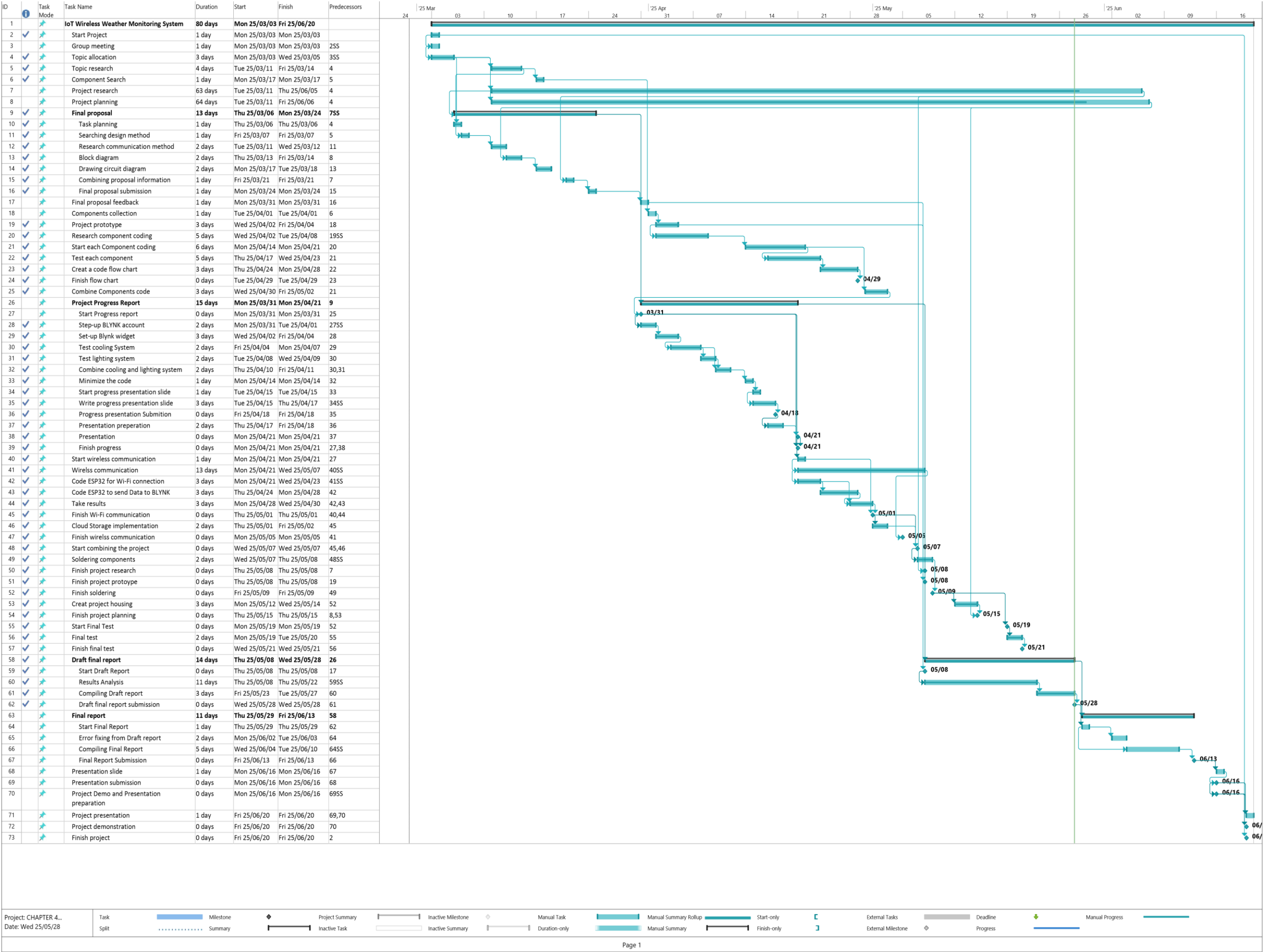
// MODE SWITCH: V3
BLYNK_WRITE(VIRTUAL_MODE) {
  mode = param.asInt();
  Serial.print("Mode: ");
  Serial.println(mode == 0 ? "Automatic" : "Manual");
}

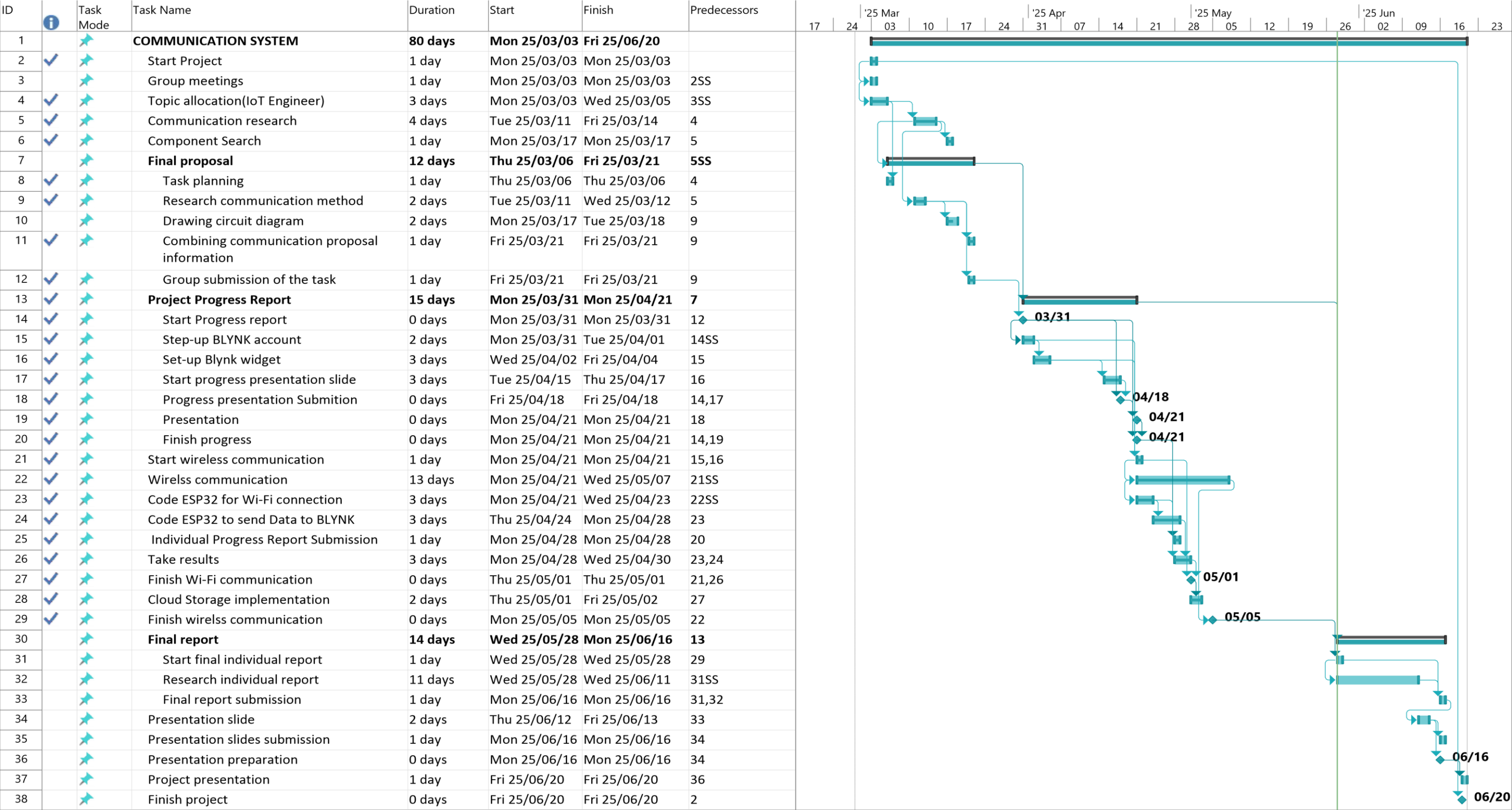
// BULB SWITCH: V4
BLYNK_WRITE(VIRTUAL_BULB) {
  manualBulb = param.asInt();
  if (mode == 1) {
    digitalWrite(LAMP_RELAY, manualBulb);
    Serial.print("Manual Bulb: ");
    Serial.println(manualBulb ? "ON" : "OFF");
  }
}

// FAN SWITCH: V5
BLYNK_WRITE(VIRTUAL_FAN) {
  manualFan = param.asInt();
  if (mode == 1) {
    digitalWrite(FAN_RELAY, manualFan);
    Serial.print("Manual Fan: ");
    Serial.println(manualFan ? "ON" : "OFF");
  }
}

```

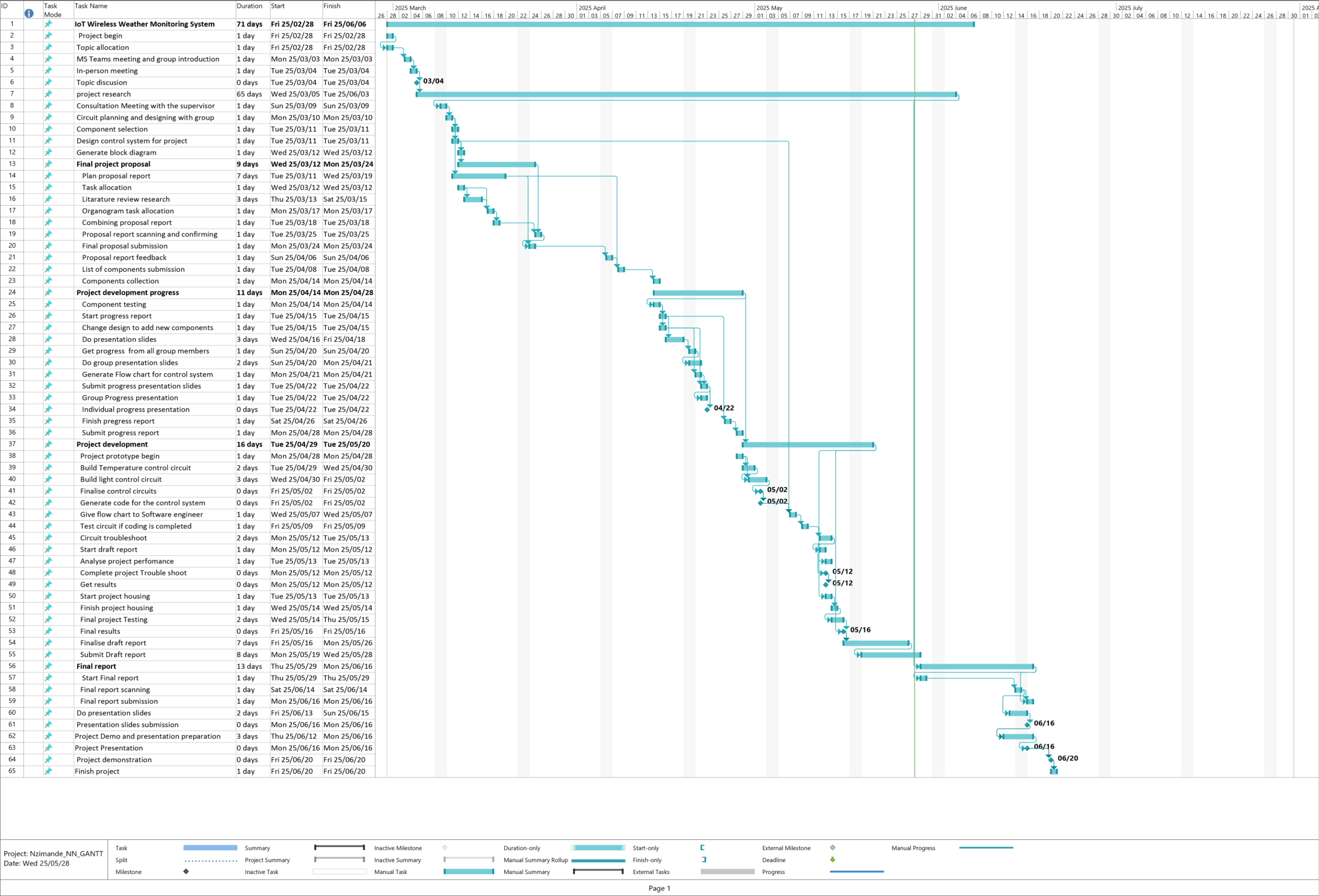
ANNEXTURE A



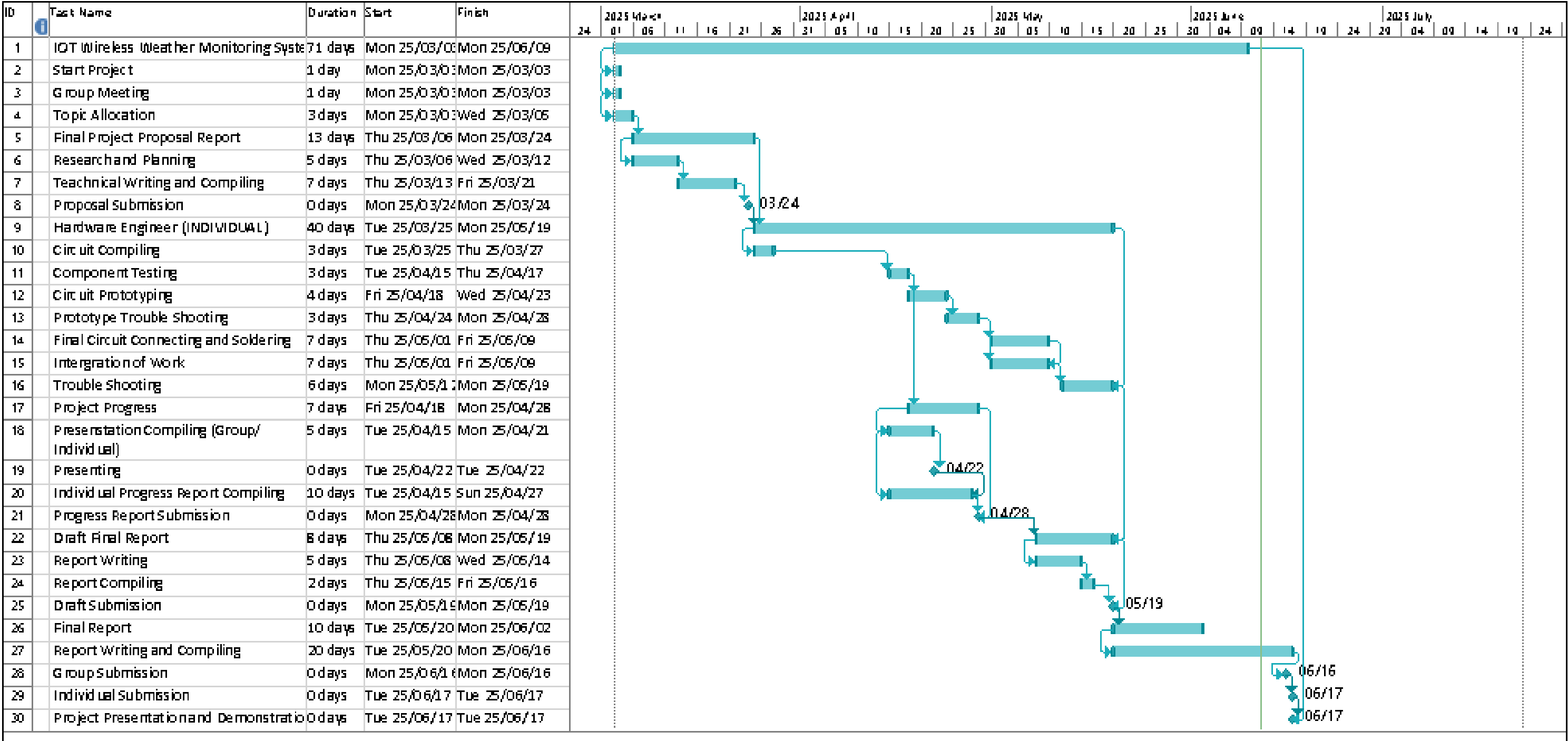


Project: CHAPTER 4... Date: Wed 25/05/28	Task		Project Summary		Manual Task		Start-only		Deadline	
	Split		Inactive Task		Duration-only		Finish-only		Progress	
	Milestone		Inactive Milestone		Manual Summary Rollup		External Tasks		Manual Progress	
	Summary		Inactive Summary		Manual Summary		External Milestone			

NZIMANDE NN_ GHANTT CHART



Padayachee D_GHANTT CHART



Ndaba S_GANTT CHART

